

Documentação de um Produto de Software - Versão 1.0

Polar - Sistema de Gestão de Ocorrências Escolares

Equipe: Islan Max dos Santos Costa e José Diôgo Oliveira da Silva

Turma: 3ºB Desenvolvimento de Sistemas

Data: 22/04/2026

Prefácio	4
1. INTRODUÇÃO AO DOCUMENTO	5
1.1. TEMA	5
1.2. OBJETIVO DO PROJETO	5
1.3. DELIMITAÇÃO DO PROBLEMA	5
1.4. JUSTIFICATIVA DA ESCOLHA DO TEMA	5
1.5. MÉTODO DE TRABALHO	6
1.6. ORGANIZAÇÃO DO TRABALHO	6
1.7. GLOSSÁRIO	6
2. DESCRIÇÃO GERAL DO SISTEMA	6
2.1. DESCRIÇÃO DO PROBLEMA	6
2.2. PRINCIPAIS ENVOLVIDOS E SUAS CARACTERÍSTICAS	7
2.2.1. Usuários do Sistema	7
2.2.2. Desenvolvedores do Sistema	7
2.3. REGRAS DE NEGÓCIOS	7
3. REQUISITOS DO SISTEMA	8
3.1. REQUISITOS FUNCIONAIS	8
3.2. REQUISITOS NÃO-FUNCIONAIS	8
3.3. PROTÓTIPO	8
3.4. MÉTRICAS E CRONOGRAMAS	9
4. ANÁLISE E DESIGN	9
4.1. ARQUITETURA DO SISTEMA	9
4.2. MODELO DO DOMÍNIO	10
4.3. DIAGRAMAS DE INTERAÇÃO	10
4.3.1. Diagrama de Sequência (descrição textual)	10
4.3.2. Diagrama de Colaboração / Comunicação (descrição textual)	11
4.4. DIAGRAMA DE CLASSES	11
4.5. DIAGRAMA DE ATIVIDADES	11
4.6. ARQUITETURA DO SISTEMA	11
4.7. DIAGRAMA DE ESTADOS	11
4.8. MODELO DE DADOS	12
4.8.1. Modelo lógico da base de dados	12
4.8.2. Criação física do modelo de dados	12
4.8.3. Dicionário de dados	12
4.9. AMBIENTE DE DESENVOLVIMENTO	13
4.10. SISTEMAS E COMPONENTES EXTERNOS UTILIZADOS	13

5. IMPLEMENTAÇÃO	13
6. TESTES	14
6.1. PLANO DE TESTES	14
6.2. EXECUÇÃO DO PLANO DE TESTES	14
7. IMPLEMENTAÇÃO	15
7.1. DIAGRAMA DE IMPLANTAÇÃO	15
7.2. MANUAL DE IMPLANTAÇÃO	15
8. MANUAL DO USUÁRIO	15
8.1. Professor	15
8.2. Coordenação	15
8.3. Direção	16
9. CONCLUSÕES E CONSIDERAÇÕES FINAIS	16
BIBLIOGRAFIA	16
GLOSSÁRIO	17

Prefácio

O projeto POLAR nasceu da observação de um problema recorrente no ambiente escolar: a perda de informações importantes relacionadas ao acompanhamento disciplinar e pedagógico dos alunos. Em muitos casos, ocorrências registradas por professores acabam dispersas, esquecidas ou sem continuidade entre os setores responsáveis, comprometendo a comunicação interna e dificultando tomadas de decisão consistentes.

A proposta do sistema é centralizar, organizar e rastrear essas informações de forma estruturada, permitindo que professores, coordenadores e diretores acompanhem o histórico completo de ocorrências, faltas, notas e registros relacionados aos alunos. O objetivo não é apenas registrar eventos, mas criar um fluxo confiável de acompanhamento institucional.

Inicialmente desenvolvido como Projeto de Conclusão de Curso (PCC), o POLAR foi concebido com uma visão além do ambiente acadêmico, buscando uma arquitetura escalável e adaptável para futuras aplicações em redes escolares maiores e possíveis integrações com sistemas educacionais já existentes.

Durante o desenvolvimento, o projeto passou por processos contínuos de modelagem, testes, validação de fluxo, organização de governança e estruturação técnica, envolvendo diferentes setores e áreas de atuação, como back-end, front-end, banco de dados, versionamento e documentação.

Este documento reúne as definições, estruturas, decisões e direcionamentos adotados ao longo da construção do projeto, servindo como base técnica e organizacional para sua evolução.

1. INTRODUÇÃO AO DOCUMENTO

Este documento apresenta a definição base do sistema Polar, um software web destinado ao registro e acompanhamento de ocorrências escolares. O conteúdo foi estruturado para servir como referência de planejamento, modelagem e implementação do projeto, em consonância com as necessidades reais da escola e com a futura possibilidade de evolução para um produto escalável.

1.1. Tema

Desenvolvimento de um sistema web para gestão de ocorrências escolares, com rastreabilidade, controle por perfis de acesso e histórico institucional completo.

1.2. Objetivo do Projeto

O objetivo geral é centralizar o registro e a tramitação de ocorrências relacionadas a alunos, substituindo o processo informal hoje baseado em comunicação dispersa. Como objetivos específicos, o projeto busca: registrar ocorrências com descrição detalhada; permitir consulta por professores, coordenação e direção; controlar o fluxo de análise e encerramento; manter histórico auditável; e fornecer uma base técnica para evolução futura do produto.

1.3. Delimitação do Problema

O projeto delimita-se ao contexto escolar, com foco em ocorrências disciplinares e pedagógicas envolvendo alunos. Não contempla, neste momento, módulos financeiros, acadêmicos amplos, comunicação com responsáveis, múltiplas instituições ou automações complexas. O recorte central é a perda de comunicação entre professores e coordenação ao registrar faltas, atrasos, desrespeito, má conduta e ausência de atividades, o que resulta em ocorrências esquecidas, sem continuidade e sem consequência institucional.

1.4. Justificativa da Escolha do Tema

A escolha do tema decorre de um problema prático observado na escola: professores registram ocorrências, mas elas se perdem entre setores e não chegam a uma conclusão formal. Isso compromete a autoridade pedagógica, a organização institucional e a análise do comportamento do aluno ao longo do tempo. Do ponto de vista acadêmico, o projeto é pertinente por envolver levantamento de requisitos, modelagem de dados, controle de acesso, arquitetura web e desenvolvimento incremental. Além disso, há perspectiva de evolução futura do sistema para uso em rede institucional maior, caso o produto seja validado e refinado.

1.5. Método de Trabalho

O desenvolvimento seguirá um processo iterativo e incremental, com orientação a objetos e modelagem baseada em UML. A abordagem é inspirada em práticas do RUP, mas adaptada à realidade do PCC. O trabalho será dividido em fases: levantamento e validação dos requisitos, modelagem do domínio, definição da arquitetura, implementação do núcleo funcional, testes e preparação para implantação. O foco inicial é o MVP funcional, sem excesso de abstração ou complexidade técnica prematura.

1.6. Organização do Trabalho

O documento está organizado da seguinte forma: o Capítulo 1 apresenta a introdução, tema, objetivo, delimitação, justificativa, método e glossário; o Capítulo 2 descreve o problema, os envolvidos e as regras de negócio; o Capítulo 3 reúne requisitos funcionais, não funcionais, protótipo e cronograma; o Capítulo 4 apresenta a análise e o design; o Capítulo 5 trata da implementação; o Capítulo 6 define testes; o Capítulo 7 aborda a implantação; o Capítulo 8 apresenta o manual do usuário; o Capítulo 9 traz as conclusões; ao final, são listadas a bibliografia e o glossário.

1.7. Glossário

Os termos essenciais do projeto são definidos de forma padronizada ao longo do documento. Entre os termos centrais estão: ocorrência, status, prioridade, histórico, coordenação, direção, professor, aluno, categoria e auditoria. O glossário completo consta no final deste documento.

2. DESCRIÇÃO GERAL DO SISTEMA

2.1. Descrição do Problema

O problema central é a desorganização do fluxo de comunicação escolar quando ocorre uma situação disciplinar ou pedagógica envolvendo um aluno. Atualmente, um professor registra a ocorrência de forma informal, mas o registro pode ser esquecido, perdido ou não tratado pela coordenação. O impacto é direto: o aluno não recebe encaminhamento adequado, a escola perde histórico e a informação não é recuperada de forma confiável. A solução proposta é um sistema digital que permita registrar, acompanhar e concluir cada ocorrência dentro de um fluxo institucional definido.

2.2. Principais Envolvidos e suas Características

2.2.1. Usuários do Sistema

O sistema destina-se ao ambiente escolar, especialmente a professores, coordenadores, vice-coordenadores, diretores e vice-diretores. Os professores registram ocorrências e acompanham o status. A coordenação analisa, encaminha e propõe solução. A direção emite o veredito final e conclui o processo. O sistema deve ser intuitivo, rápido e adequado a usuários com diferentes níveis de familiaridade tecnológica.

2.2.2. Desenvolvedores do Sistema

O desenvolvimento é realizado por estudantes do curso de Desenvolvimento de Sistemas, com apoio de orientadores e revisão dos documentos por membros da equipe responsáveis por documentação, implementação, modelagem e validação. Como o projeto possui caráter acadêmico, a distribuição de responsabilidade deve ser clara para reduzir retrabalho e garantir consistência entre documentação e código.

2.3. Regras de Negócio

- Toda ocorrência deve estar vinculada a um aluno.
- Toda ocorrência deve possuir um autor identificado.
- Toda ocorrência nasce com status REGISTRADA.
- A descrição da ocorrência deve ser detalhada, em formato de relatório.
- O professor registra e consulta; não altera status.
- A coordenação analisa e resolve; o diretor encerra.
- Toda alteração relevante gera histórico auditável.
- Comentários separados não existem; a rastreabilidade é centralizada no histórico.
- Ocorrências encerradas não podem ser editadas.
- O sistema deve preservar o histórico completo do aluno para consulta futura.

3. REQUISITOS DO SISTEMA

3.1. Requisitos Funcionais

Os requisitos funcionais descrevem as ações que o sistema deve executar. A lista abaixo representa o escopo mínimo do MVP.



3.3.1. Diagrama de Navegação (descrição textual)

A sequência de navegação prevista é: Login -> Dashboard -> Cadastro de ocorrência -> Detalhes da ocorrência -> Histórico do aluno -> Dashboard. O usuário pode retornar ao painel principal a qualquer momento. A coordenação e a direção podem acessar, a partir da tela de detalhe, opções de alteração de status conforme permissão.

3.4. Métricas e Cronograma

Para o PCC, recomenda-se uma estimativa simples por entregas incrementais. Considerando o escopo atual, o esforço total pode ser distribuído em aproximadamente cinco semanas de execução principal, mais uma semana de estabilização. Em termos de funcionalidade, o núcleo do produto concentra autenticação, cadastro básico, ocorrências, workflow, histórico e interface.



4. ANÁLISE E DESIGN

4.1. Arquitetura do Sistema

A arquitetura adotada é web em três camadas, com separação entre apresentação, aplicação e persistência. O frontend será hospedado em GitHub Pages como aplicação estática. A comunicação entre backend e frontend será feita via API em js, responsável por autenticação, regras de negócio e acesso aos dados.

4.2. Modelo do Domínio

O modelo do domínio é centrado na entidade Ocorrência. As entidades principais são: User, Role, Student, Class, Occurrence, HistoryLog e Category. O relacionamento básico é o seguinte: um usuário pertence a um papel; um usuário registra ocorrências; um aluno pertence a uma turma; uma ocorrência pertence a um aluno e possui histórico; o histórico registra cada alteração de estado e cada decisão tomada. A ocorrência é o núcleo de integração de todo o sistema.



4.3. Diagramas de Interação

4.3.1. Diagrama de Sequência (descrição textual)

Fluxo de registro de ocorrência: o professor autentica-se, acessa o formulário, informa o aluno, descreve o fato, seleciona categoria e prioridade, e envia os dados. O backend valida permissões, persiste a ocorrência com status inicial REGISTRADA e grava um HistoryLog. Em seguida, a interface atualiza a lista de ocorrências. Fluxo de mudança de status: o coordenador ou diretor abre a ocorrência, solicita a alteração de estado, o backend valida o perfil e a transição permitida, atualiza o status e insere um registro no histórico. Se a transição for inválida, o sistema retorna erro e não altera o banco.

4.3.2. Diagrama de Colaboração / Comunicação (descrição textual)

Os objetos centrais da interação são: Usuário, Interface, API, Serviço de Ocorrências, Serviço de Histórico e Banco de Dados. A comunicação ocorre em cadeia: Interface envia requisição -> API recebe -> Serviço aplica regra -> Banco persiste -> Histórico é registrado -> resposta retorna para a Interface.

4.4. Diagrama de Classes

A implementação inicial pode ser representada pelas seguintes classes: User(id, name, email, passwordHash, role, createdAt); Role(id, name); Class(id, name, year); Student(id, name, classId, createdAt); Category(id, name, description, active); Occurrence(id, studentId, categoryId, priority, description, status, createdBy, createdAt, updatedAt); HistoryLog(id, occurrenceId, action, previousStatus, newStatus, performedBy, timestamp). Métodos principais: authenticate(), createOccurrence(), changeStatus(), addHistory(), listOccurrences(), getStudentHistory().

4.5. Diagrama de Atividades

O fluxo de atividades inicia com o login do usuário. Em seguida, o professor seleciona o aluno, preenche a ocorrência e envia. O sistema valida dados, grava o registro e retorna ao painel. A coordenação acessa a ocorrência, analisa o conteúdo e encaminha a resolução. A direção verifica o resultado e encerra a ocorrência. Cada transição atualiza o histórico.

4.6. Diagrama de Estados

O ciclo de vida da ocorrência é composto pelos estados REGISTRADA, EM_ANALISE, RESOLVIDA e ENCERRADA. A ocorrência inicia em REGISTRADA, passa para EM_ANALISE quando a coordenação assume o caso, evolui para RESOLVIDA quando há encaminhamento definido e termina em ENCERRADA quando a direção conclui formalmente o processo. Após ENCERRADA, o registro torna-se somente leitura.

4.7. Diagrama de Componentes

Os componentes principais são: Frontend Web, API Backend, Módulo de Autenticação, Módulo de Ocorrências, Módulo de Histórico, Módulo de Cadastro e Banco de Dados. O frontend consome a API; a API orquestra serviços internos; os serviços manipulam persistência; o banco armazena os dados de forma relacional.

4.8. Modelo de Dados

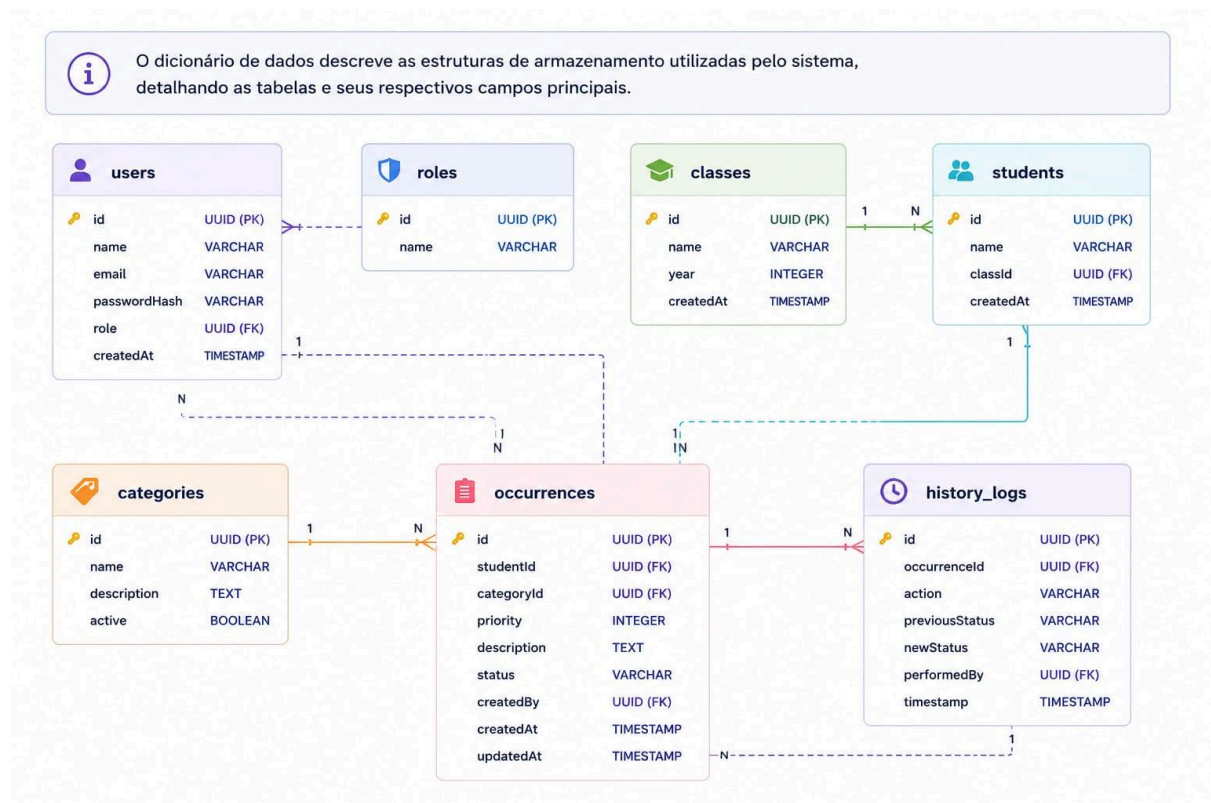
4.8.1. Modelo Lógico da Base de Dados

O modelo lógico é relacional e normalizado. As tabelas principais previstas são: users, roles, classes, students, categories, occurrences e history_logs. A tabela occurrences possui chave estrangeira para students, categories e users. A tabela history_logs referencia occurrences e users. A estrutura evita duplicação de dados e permite consulta histórica eficiente.

4.8.2. Criação Física do Modelo de Dados

A criação física será feita via migrations do PostgreSQL e Supabase. Campos de texto, datas, chaves estrangeiras e índices devem ser definidos para garantir integridade e desempenho básico. A exclusão física de ocorrências não deve ser utilizada; recomenda-se preservação do histórico e, se necessário, marcação lógica de status.

4.8.3. Dicionário de Dados



4.9. Ambiente de Desenvolvimento

- Editor de código: Visual Studio Code.
- Frontend: HTML, CSS, JavaScript e React.
- Backend: Python e JavaScript.
- Framework backend: Python (engine) e API rest.
- Banco de dados: PostgreSQL.
- Serviços de banco e autenticação: Supabase.
- Controle de versão: Git e GitHub.

4.10. Sistemas e Componentes Externos Utilizados

O projeto utiliza serviços e componentes externos de baixo custo ou gratuitos, adequados ao contexto acadêmico: GitHub para repositório e pages; Render ou serviço equivalente para backend; bibliotecas de autenticação e validação; e eventualmente ícones ou componentes visuais leves. Não há integração obrigatória com sistemas pagos nesta fase.

5. IMPLEMENTAÇÃO

A implementação deve seguir a modelagem definida nas seções anteriores, com separação entre rotas, controladores, serviços, validações e acesso ao banco. O backend será organizado em módulos para autenticação, usuários, alunos, turmas, ocorrências e histórico. A lógica de negócio deve permanecer no servidor, não no frontend.

- Padronização de nomes de arquivos, funções, tabelas e variáveis.
- Uso de validação de entradas antes de gravar dados.
- Tratamento explícito de erros e respostas padronizadas
- Registro automático de cada mudança relevante no histórico.
- Proteção de rotas por autenticação e perfil.
- Interface simples, com telas objetivas e sem excesso de elementos.

6. TESTES

6.1. Plano de Testes



6.2. Execução do Plano de Testes

A execução do plano de testes será realizada após a implementação das funcionalidades principais. O ambiente de teste deve registrar o nome do responsável, a versão do sistema, o navegador utilizado e o resultado obtido em cada caso. Enquanto o sistema estiver em desenvolvimento, este item deve ser atualizado com os resultados reais de cada teste executado. Exemplo de registro de execução: Sistema Polar, versão 1.0; responsável pelos testes: equipe de desenvolvimento; ambiente: navegador Chrome em desktop; observação: testes críticos de autenticação, criação de ocorrência e controle de acesso devem ser priorizados primeiro.

7. IMPLANTAÇÃO

7.1. Diagrama de Implantação

A implantação prevista utiliza o modelo cliente-servidor. O frontend estático é hospedado no GitHub Pages e executado no navegador do usuário. O backend é hospedado em um serviço gratuito compatível com as API's utilizadas. O banco de dados Supabase fica em serviço separado. O fluxo físico é: navegador -> frontend -> API backend -> banco de dados.

7.2. Manual de Implantação

- Publicar o frontend no GitHub Pages a partir da branch principal ou de uma branch dedicada.
- Configurar a URL da API por variável de ambiente.
- Subir o backend em serviço gratuito compatível com Node.js.
- Criar o banco de dados PostgreSQL e aplicar as migrations.
- Validar autenticação, CRUD de ocorrências e consulta de histórico.
- Realizar um teste final com usuário real antes da demonstração.

8. MANUAL DO USUÁRIO

O sistema Polar foi projetado para ser usado com poucos cliques e sem treinamento extenso. A seguir estão as instruções básicas por perfil.

8.1. Professor

- Acessar a tela de login.
- Informar e-mail e senha.
- Acessar o painel principal.
- Selecionar o aluno.
- Preencher a descrição detalhada da ocorrência.
- Definir categoria e prioridade.
- Enviar o registro.
- Consultar depois o status e o histórico do aluno.

8.2. Coordenação

- Entrar com credenciais institucionais.
- Abrir a lista de ocorrências em análise.
- Ler o relatório completo.
- Alterar o status quando necessário.
- Registrar o encaminhamento institucional.
- Conferir se a ação foi gravada no histórico.

8.3. Direção

- Acessar ocorrências resolvidas ou em espera de validação.
- Verificar o histórico completo.
- Efetuar a conclusão formal.
- Confirmar que o caso ficou encerrado e somente leitura.

9. CONCLUSÕES E CONSIDERAÇÕES FINAIS

O Polar nasce como resposta a um problema real da escola e, ao mesmo tempo, como projeto acadêmico com potencial de evolução. A proposta é transformar registros dispersos de ocorrências em um fluxo digital coerente, rastreável e útil para professores, coordenação e direção. O escopo foi deliberadamente mantido enxuto para viabilizar a entrega do PCC, sem impedir crescimento futuro.

Como limitação, o projeto depende da disciplina da equipe, da consistência das decisões técnicas e da manutenção do escopo. Como oportunidade, ele pode evoluir para um produto institucional mais amplo, com integração a redes de sistemas educacionais. Em sua forma atual, o maior valor do Polar é validar uma solução útil, objetiva e demonstrável, com potencial real de continuidade.

BIBLIOGRAFIA

- PRESSMAN, Roger S. Software Engineering: A Practitioner's Approach.
- SOMMERVILLE, Ian. Engenharia de Software.
- BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML - Guia do Usuário.
- FOWLER, Martin; SCOTT, Kendall. UML Essencial.
- Documentação de Produto de Software, Profa. Ana Paula Gonçalves Serra.
- Documentação e requisitos institucionais fornecidos pela equipe do projeto Polar.

GLOSSÁRIO

- **Ocorrência:** Registro formal de um fato disciplinar ou pedagógico envolvendo um aluno.
- **Status:** Estado atual da ocorrência dentro do fluxo institucional.
- **Prioridade:** Indicador do nível de urgência ou gravidade da ocorrência.
- **Histórico:** Linha do tempo com todas as ações relevantes registradas.
- **Coordenação:** Grupo responsável por analisar e encaminhar ocorrências.
- **Direção:** Instância responsável pela validação final e encerramento.
- **Aluno:** Pessoa vinculada à ocorrência e afetada pelo processo institucional.
- **Categoria:** Classificação do tipo de ocorrência.
- **Auditoria:** Capacidade de rastrear quem fez o quê, quando e em qual contexto.
- **MVP:** Versão mínima funcional do sistema para validação prática.
- **PCC:** Projeto de Conclusão de Curso.
- **Frontend:** Parte visual utilizada pelo usuário no navegador.
- **Backend:** Parte lógica e de processamento do sistema.
- **API:** Interface de comunicação entre frontend e backend.